```
  case cards
  in [*, {rank: "Ace", suit: String => s}, *]
    "You have an Ace! Its suit is #{s}."
  in [*, {rank: r, suit: "Diamonds"}, *]
    "You have a Diamond! Its rank is #{r}."
  in [*, {rank: "Queen", suit:}, *]
    "You have a Queen! Its suit is #{suit}."
  else
    "You have no interesting cards,"
  end
end

puts pick_a_card([
  {rank: "Ace", suit: "Hearts"},
  {rank: "King", suit: "Diamonds"},
  {rank: "Queen", suit: "Clubs"}
])
```

produces:

```
You have an Ace! Its suit is Hearts.
```

The case/in statement works like a successive set of pattern matchings the way we've already seen pattern matches work. The target variable, in this case our list of card hashes that is passed to cards, is matched against the first pattern. The first pattern is a find pattern, matching against the first hash with key rank and value Ace. The rest of the pattern assigns the variable in the suit key to s, and then that variable is local to all the code within that pattern.

If that pattern does not match, the next one is tried. In this case the second pattern looks for a suit of Diamonds, and we're not expecting anything in particular for the matching rank, so we can just bind it to r. If that doesn't match, the third line shows another shortcut for variable assignment, in this one the hash key shortcut is used to assign suit to the suit of the card that matches rank: "queen".

Eventually, we run out of patterns. If there's an else clause, then the else clause is executed. If not, the statement raises a NoMatchingPatternError. The expectation is that case statement will be complete and always have a clause to execute, even if that clause is the else clause.

A case statement either has in clauses or it has when clauses, you cannot mix the two in a single statement.

Pinning Values

Pattern matching is a great way to combine validating data with variable assignment, but we are still missing an important piece of the puzzle.

In our earlier card example, the pick_a_card example hard codes the ranks of cards it is looking for. But what if you wanted to use an existing value, for example if you wanted to pass a value to look for?

```
def pick_a_card(rank_to_look_for, suit_to_look_for, cards)
  case cards
  in [*, {rank: ^rank_to_look_for, suit:}, *]
    "You have a #{rank_to_look_for}! Its suit is #{suit}."
  in [*, {rank:, suit: ^suit_to_look_for}, *]
    "You have a {rank}! Its suit is #{suit_to_look_for}."
  else
```